

TGPrint: Attack fingerprint classification on encrypted network traffic based graph convolution attention networks

Leiqi Wang^{a,b}, Xiu Ma^{a,b}, Ning Li^a, Qiujian Lv^{a,*}, Yan Wang^{a,b}, Weiqing Huang^{a,b}, Haiyan Chen^c

^a Institute of Information Engineering, Chinese Academy of Science, Beijing, China

^b School of Cyber Security, University of Chinese Academy of Science, Beijing, China

^c Chinese Research Academy of Environmental Sciences, Beijing, China

ARTICLE INFO

Keywords:

Attack classification
Encrypted network traffic
Unseen attack
Attack graph
Graph neural networks

ABSTRACT

Nowadays, most network traffic is encrypted, which protects user privacy but hides attack traces, further hindering identifying attacks to inspect traffic packages. Machine Learning (ML) methods are widely applied to attack classification on encrypted traffic owing to no need for manual analysis. However, existing studies only concentrate on basic statistical features and cannot obtain the crucial attack behaviors hiding in the encrypted traffic. Worse still, attackers constantly update attack vectors to evade detection, which means outdated features extracted from historical traffic fail to recognize unseen attacks.

As a solution, we propose an attack classification approach, attack fingerprint based on graphs of time-window (TGPrint). We first filter normal traffic flows using ML models to eliminate the impact of useless, noisy data for attack classification and maintain suspicious traffic. Then, we create attack graphs to depict interaction behaviors of attack-victim hosts from suspicious traffic containing crucial attack behaviors. Besides, we divide a specific duration for each attack to precisely elaborate attack graphs, where temporal, statistical, and aggregate features are extracted to portray attack behaviors. Finally, we utilize Graph Neural Networks (GNNs) to mine and grasp the crucial behavior patterns from attack graphs to generate fingerprints and classify attacks, even unseen attacks. Extensive experiments are conducted on well-known datasets to verify our approach. It achieves a precision of 99% in attack classification on encrypted traffic, an average higher than other ML methods of 50%. Meanwhile, it classifies unseen attacks with an average accuracy of over 80% and has a strong robustness to false positives.

1. Introduction

Guaranteeing those online activities and the sensitive information exchanged daily remain private, Transport Layer Security (TLS) protocol was created in 1999 (Dierks, 1999) which provides encryption for the application layer (Roques et al., 2019). Gartner estimated that 80% of global web traffic would be encrypted by 2019 (Cisco, 2019). However, privacy does not guarantee security. Attackers have started to leverage TLS to hide their persistent malicious activities. Pattern-matching techniques are hard to recognize attacks in encrypted traffic, such as port identification and Deep Packet Inspection (DPI). Port-based approaches encounter failures in identifying attacks due to dynamic port techniques (Madhukar and Williamson, 2006; Erman et al., 2007).

DPI has been proven inefficient in attack classification tasks because it directly goes against the purpose of encryption protocol, which is to keep user data confidential. Therefore, how to detect and reveal attack behaviors based on encrypted traffic has become a vital research hot spot.

Given the above constraints, ML on the encrypted network for attack classification is a natural solution without complicated manual analysis. The application of ML to recognize attacks in encrypted network traffic classification has been well studied in the past decades (Luo et al., 2020; Zhai et al., 2020; Nychis et al., 2008; Koch et al., 2014; Torroledo et al., 2018a; Yao et al., 2019). Unfortunately, the promise of ML has been slow to materialize in the network security domain (Anderson and

* Corresponding author.

E-mail addresses: wangleiqi@iie.ac.cn (L. Wang), maxiu@iie.ac.cn (X. Ma), lining@iie.ac.cn (N. Li), lvqiujian@iie.ac.cn (Q. Lv), wangyan@iie.ac.cn (Y. Wang), huanweiqing@iie.ac.cn (W. Huang), chenhy@craes.org.cn (H. Chen).

<https://doi.org/10.1016/j.cose.2023.103466>

Received 30 January 2023; Received in revised form 22 August 2023; Accepted 31 August 2023

Available online 12 September 2023

0167-4048/© 2023 Elsevier Ltd. All rights reserved.

McGrew, 2017). There are three main challenges for ML-based attack classification approaches.

Challenge 1: Imprecise features without attack behavior patterns. The sophisticated features decide the model's detection effectiveness because of *garbage in, garbage out*. Features in attack classification studies on encrypted traffic are mainly classified into two aspects: 1) statistical and temporal traffic features (e.g., packet size, traffic direction, transmission time) (Gu et al., 2008; Arora et al., 2014; Anderson et al., 2018) and 2) TLS handshake features (e.g., handshake message status, encryption suite). The former focuses on the detailed transmission information, and the latter grasp TLS session details.

An attack lasts between attack-victim hosts, while attack behaviors leave traces in the traffic. However, these two features, exclusive of the interaction information, cannot be abstracted to crucial high-level attack behavior patterns. The attack behavior patterns can enhance the models' capability of attack classification on encrypted traffic because they reflect the attacks' behavior characteristics from interaction activities. The interaction information of hosts' traffic restores the attack behavior patterns, which are crucial for attack classification.

Challenge 2: Unseen attack with new attack vectors. Various attack vectors can implement an attack. For example, brute-force attack vectors involve a variety of protocols, such as HTTP and FTP. Different attack vectors execute different operations during the attack, and the malicious traffic generated from diverse attack operations is disparate. ML models learn traffic characteristics based on the attack already seen and make accurate judgments when the same attack reappears. The traffic the new attack vector generates is different from the known, making the model misjudge. Hence, attackers usually change the attack vectors, which makes the classification model confused and unable to detect the attack activities. Therefore, the ability to identify unseen attacks with new attack vectors is a critical assessment criterion in addition to the evaluation, such as accuracy. This paper defines an unseen attack as a known attack with a new attack vector.

To address the above challenges, we need to propose an approach. It can mine the crucial attack behavior patterns in encrypted traffic for accurate attack classification. Meanwhile, it identifies unseen attacks based on these crucial patterns, which reveal fundamental techniques for the same attack with different attack vectors. The emergence of GNN has demonstrated the capability of learning complex behavior and extracting high-level features from big data to create the signature or fingerprint of analyzed data (Pham et al., 2021; Kipf and Welling, 2016; Veličković et al., 2017). High-level attack behaviors with complex interactions among hosts could be graph-structured for recognizing attacks. Inspired by this, we propose an attack classification approach TGPrint, which uses attack graphs as a fingerprint for attack classification on encrypted traffic. TGPrint first eliminates noise from normal traffic flows using ML tools and leaves suspicious traffic to construct attack graphs. Then, TGPrint clusters attack-victim hosts' traffic and divides them according to a specific time window set for each attack to portray attack behaviors as a graph precisely. Thirdly, TGPrint constructs a set of attack graphs where ports are nodes, attack-victim hosts' communication relationships are edges, and temporal, statistical, and aggregate features are edge attributes to generate each attack's fingerprint. Finally, TGPrint exploits GCAN's learning ability of graph representation to generate fingerprints and mines crucial attack behavior patterns to classify attacks.

In summary, the main contributions of this paper are as follows:

- We propose TGPrint to classify attacks precisely in encrypted traffic based on attack fingerprints, which obtain the high-level crucial attack behavior patterns.
- We use GCAN to mine behavior patterns from attack graphs to identify unseen attacks and reveal fundamental techniques of the attack, which adapt to the behaviors' changing over time.

- We construct attack graphs to precisely depict interaction behaviors of attack-victim hosts with a time window ratio τ to divide a specific duration for each attack's encrypted traffic.
- We run extensive experiments on three well-known datasets to demonstrate the effectiveness of TGPrint, which achieves a precision of 99% in attack classification on encrypted traffic, an average higher than other ML methods of 50% and achieves an average accuracy of over 80% in unseen attacks classification.

2. Related work

2.1. Attack classification

2.1.1. Pattern-matching approaches

Pattern-matching approaches focus on manually set rules to look for the particular malicious program or the data patterns reflected from the packet traffic. Port mapping classifies traffic by mapping the port in the TCP/UDP message header to the specific attack. However, some applications use dynamic ports (Madhukar and Williamson, 2006), which may change during data transmission; some applications use ports of other standard protocols for data transmission in order to avoid system restrictions (Erman et al., 2007), thus achieving port steganography. DPI (Park et al., 2013) parses the network packets as a whole during the traffic analysis process and determines whether the traffic in the packets matches the characteristics of a specific attack. However, DPI only applies to plaintext traffic and has a high computational overhead. These approaches are no longer suited for attack classification in encrypted traffic. TLS certificates sent by web servers in plaintext can also help detect phishing websites or anomalous connections (Richman, 2022). Blacklists maintaining certificates from malicious servers and checking all incoming certificates against that list are the easiest way of dealing with certificates (Suricata, 2010; Abuse, 2022). However, this method must be updated regularly by trusted people, and mistakes resulting in blocked websites can frustrate users. These approaches are no longer suited for attack classification in encrypted traffic.

2.1.2. Machine learning approaches

ML algorithms have been the critical research direction in recent years, whose main idea is to determine what attack there is by analyzing the encrypted network traffic features (Anderson and McGrew, 2017; FRANTISEK S, 2017; Alqudah and Yaseen, 2020; Lopez-Martin et al., 2017; Torroledo et al., 2018b; Duessel et al., 2017). Anderson (Anderson and McGrew, 2017) demonstrates six ML algorithms perform well with malicious traffic classification when confronted with real network data. Frantisek (FRANTISEK S, 2017) compares four ML methods, XGBoost, Random Forest, Neural Network, and SVM, for TLS encrypted malicious traffic detection. They conclude that XGBoost and Random Forest methods are effective, and the accuracy of XGBoost and Random Forest is about 95%. Besides, Deep Learning (DL), a new branch of ML, is also fully applied to attack classification on encrypted traffic. Ivan (Torroledo et al., 2018b) analyze and summarize forty features in four classes for detecting malware and phishing signatures using LSTM and obtain 94.87% and 88.64% accuracy, respectively.

However, feature processing is a significant challenge in ML approaches, which can directly affect the network's classification performance and require much work. How to extract and represent the limited features in the encrypted traffic are the key to identifying attacks accurately.

2.1.3. Deep learning's challenges

Despite its effectiveness, deep learning also has some limitations when applied to attack identification. Deep learning models require a large amount of labeled data for training (Mohamad Tahir et al., 2015). However, in the field of cyber security, obtaining labeled attack data can be challenging due to the limited availability of comprehensive and diverse datasets. Moreover, the distribution of attacks is often

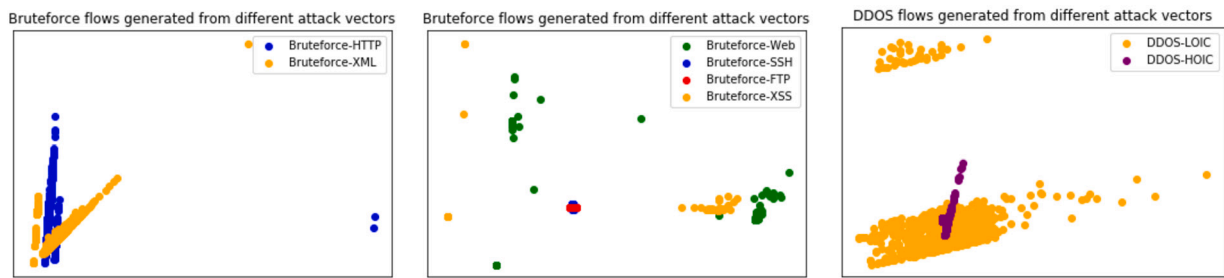


Fig. 1. Traffic distribution is different in the attack with various attack vectors in HIKARI and CICIDS. Left one is from HIKARI and the other two are from CICIDS. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

highly imbalanced, with rare attacks being significantly outnumbered by normal network traffic. On the other hand, deep learning models may struggle to generalize well to previously unseen attack types or variations (Chou and Jiang, 2021a). This limitation arises from the reliance on labeled training data, as the models may not have learned the relevant features or patterns necessary to detect novel attacks.

2.2. Unseen attack classification

Coping to recognize unseen attacks is particularly important in network security (Popoola et al., 2021; Duessel et al., 2017; Hindy et al., 2020; Bar and Hajaj, 2022; Sarhan et al., 2021), where new attack vectors are constantly emerging and continue to break through cyber security defenses. The low accuracy rate that occurs when identifying unseen attacks causes significant damage to network security. Sarhan (Sarhan et al., 2021) maps the network data features using ML models to distinguish semantic attributes from seen attack classes and detect the unseen classes by constructing the relationships between known and zero-day attacks. Popoola (Popoola et al., 2021) proposes the federated DL (FDL) method for zero-day botnet attack detection to avoid data privacy leakage in IoT-edge devices. Hindy (Hindy et al., 2020) proposes an auto-encoder implementation benefiting greatly from encoding-decoding capabilities in zero-day attack detection. Bar (Bar and Hajaj, 2022) and Duessel (Duessel et al., 2017) inspired by natural language processing (NLP), incorporate syntactic context into attack detection and increases unseen attack detection accuracy. These studies work a lot on feature processing to detect unknown attacks. However, they need to focus on interaction behaviors between attack-victim hosts in an attack and find fundamental techniques.

3. Background

3.1. Attack vectors

Attack vectors are the means and methods used to achieve an attack, and various attack vectors can accomplish an attack. Different attack vectors execute different operations during the attack, and the malicious traffic generated from diverse attack operations is disparate. Fig. 1 shows the traffic distribution of Bruteforce and DDoS with different attack vectors from three datasets. Traffic is mapped into 2-dimensions to show the difference between various attack vectors of the same attack, Bruteforce and DDoS, intuitively. Bruteforce has two attack vectors, HTTP and XML, in HIKARI, and four attack vectors, SSH, FTP, Web, and XSS, in CICIDS. DDoS has two attack vectors, HOIC and LOIC, in CICIDS. The traffic distribution changes considerably when the attack vector is changed. Even if the classification model has seen the attack before, it can still not identify it when a new attack vector comes. Unlike 0day attacks, unseen attacks in this paper refer to the emergence of a new attack vector in the same type of attack. For example, in bruteforce, an XML-based implementation of the attack is an unseen attack for the original http-based one. Therefore, the ability to unseen attack recognition is essential in evaluating a model.

3.2. Graph neural networks for graph representation

Graphs, a data structure consisting of nodes and edges, provide an intuitive representation of abstract relationships and interactions. Mathematically, a graph G is defined as a tuple (V, E) where V is the set of nodes, and E is the set of edges linking to the nodes. Each node, $v \in V$ is associated with n -dimensional vectors x_v , $n \in \mathbb{N}$. With the prevalence of graph-structured data, there is an increasing need for graph representation learning (Chen et al., 2020). Graph Neural Networks (GNNs) are a practical framework for the representation learning of graphs. GNNs follow a neighborhood aggregation scheme, where the representation vector of a node is computed by recursively aggregating and transforming representation vectors of its neighboring nodes (Xu et al., 2018). Many variants of GNNs with different neighborhood aggregation and graph-level pooling schemes have been proposed. Graph isomorphism network (GIN) (Xu et al., 2018) can achieve maximum discriminative power by producing the embedding of the entire graph when given embeddings of individual nodes. Graph convolution network (GCN) (Kipf and Welling, 2016) scales linearly in the number of graph edges via a localized first-order approximation of spectral graph convolutions and learns hidden layer representations that encode both local graph structure and features of nodes. Graph Attention Networks (GAT) (Veličković et al., 2017) leverages masked self-attentional layers and specifies different weights to different nodes in a neighborhood by stacking layers to which nodes can attend their neighborhoods' features. This paper uses GIN, GCN, and GAT layers to construct the model and represent and classify the attack.

4. Approach

This paper aims to fingerprint persistent attacks on encrypted network traffic. Fig. 2 shows an overview of our approach TGPrint: we take network traffic of attack-victim hosts as input and generate fingerprints that identify attacks. After feature extraction, TGPrint first removes normal flows to eliminate the noise using ML tools and leaves suspicious traffic. Subsequently, TGPrint clusters suspicious attack-victim hosts' traffic and divides them according to a specific time window for each attack. After that, TGPrint portrays attack behaviors as a graph and constructs a set of graphs for each attack type. Then, TGPrint exploits GCAN to generate fingerprints and mines crucial attack behavior patterns. Finally, the generated fingerprints can recognize attacks precisely based on attack behavior patterns from attack graphs.

4.1. Feature processing

4.1.1. Feature select

We analyze TLS handshake information in attack traffic to select features used to construct an attack graph. Server names, TLS version, and TLS certificate are three we mainly considered (Shen et al., 2016). We first extracted the server names but found no malicious webs in them. Instead, as shown in Table 1, attacks frequently happen on standard web pages such as Google, Facebook, and cloud server websites. It clarifies whether a web page is malicious and is of little help in judging the

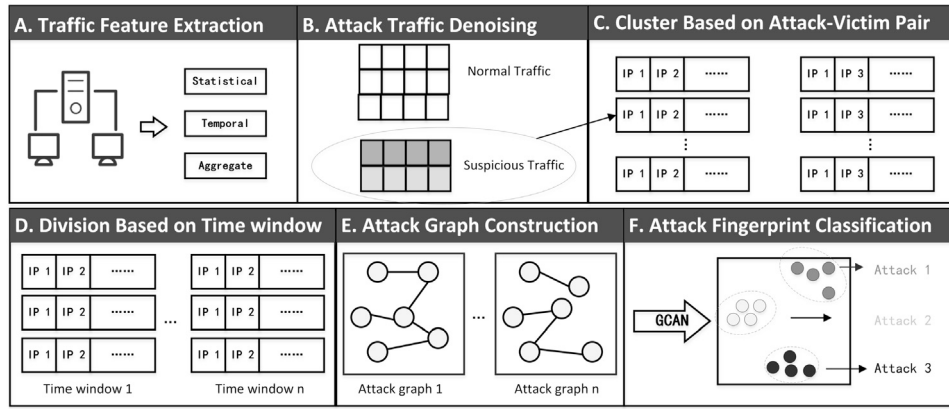


Fig. 2. Overview of the creation and matching of attack fingerprints. (A) Extract features from the network traffic. (B) Attack traffic denoising, maintaining suspicious flows. (C) Cluster the suspicious flows from two communicating hosts. (D) Divide the suspicious flows from host pairs by time window. (E) Construct attack graphs based on port nodes, communication edges, and behavior attributes. (F) Match attack fingerprints generated from attack graphs by the GCAN model.

Table 1
Rank of the Frequency in Server Name When Attacking.

Rank	Server Name	Counts
1	vic3.clouds.web.id	1216
2	www.google-analytics.com	33
3	www.facebook.com	29
4	api.autopilot.hq.com	28
5	stats.g.doubleclick.net	25
6	connect.facebook.net	24
7	static.xx.fbcdn.net	21
8	oricer.com.ec	20
9	googleads.g.doubleclick.net	20
10	hackmag.com	20

attack. Then, we extract the TLS version and TLS certificates. Since the TLS version is usually uniform, we do not consider the relationship between the TLS version and attack classification. Because checking the certificate or not is up to the policy of the application server, TLS certificates are not necessary for communication connections established on the TLS protocol. The impact of the certificate also could be ignored. Finally, we use 86 features shown in section 4.1.2, including statistical, temporal, and aggregate features.

4.1.2. Feature evaluation

Using the Mutual Information (MI), we compute the MI score of each feature with the attack label and ranked the results. Table 2 shows the forty highest-ranked features and provides all the correlation scores. The table shows that features with high MI belong to various types. Statistical features are the basic information of the payload and package during communication; temporal features are information such as transmission time; aggregate features are calculated from statistical and temporal features. Hence, we construct attack fingerprints from multiple features and utilize them to accomplish attack classification.

$$I(X;Y) = \sum_{x \in X} \sum_{y \in Y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)} \quad (1)$$

4.1.3. Attack traffic denoising

Attack classification is according to traces hidden in traffic to identify attacks. Normal traffic is noise because it is not the purpose of attack classification. Furthermore, that has far more than attack traffic, so the trained model is more sensitive to normal behavior, which violates the intention of attack classification. Meanwhile, normal behavior patterns hardly change because normal people have a fixed daily routine. Therefore, normal traffic can be identified stably, accurately, and effortlessly with a simple model.

Table 2
Features of Top 40 MI Scores.

Rank	Features	Category
1	fwd_subflow_bytes	Statistical
2	bwd_iat.tot	Aggregate
3	bwd_subflow_pkts	Statistical
4	fwd_iat.min	Aggregate
5	fwd_header_size_min	Aggregate
6	bwd_iat.max	Aggregate
7	fwd_subflow_pkts	Statistical
8	bwd_iat.min	Aggregate
9	payload_bytes_per_second	Aggregate
10	bwd_pkts.payload.tot	Statistical
11	fwd_iat.tot	Aggregate
12	flow_iat.tot	Aggregate
13	flow_iat.min	Aggregate
14	flow_iat.max	Aggregate
15	flow_iat.std	Aggregate
16	fwd_data_pkts.tot	Statistical
17	fwd_pkts.payload.std	Aggregate
18	fwd_header_size.tot	Statistical
19	down_up_ratio	Aggregate
20	bwd_iat.std	Aggregate
21	fwd_bulk_packets	Statistical
22	idle.max	Aggregate
23	fwd_iat.std	Aggregate
24	flow_pkts.payload.max	Aggregate
25	flow_pkts.payload.min	Aggregate
26	fwd_iat.max	Aggregate
27	bwd_pkts.payload.min	Aggregate
28	active.std	Aggregate
29	active.avg	Aggregate
30	idle.min	Aggregate
31	bwd_pkts.payload.max	Aggregate
32	bwd_header_size.tot	Statistical
33	bwd_bulk_bytes	Statistical
34	fwd_header_size_max	Aggregate
35	bwd_data_pkts.tot	Statistical
36	flow_duration	Temporal
37	bwd_URG_flag_count	Statistical
38	flow_CWR_flag_count	Statistical
39	fwd_URG_flag_count	Statistical
40	bwd_subflow_bytes	Statistical

However, attack evolution increases the difficulty of attack identification because of evolution uncertainty, and preset labels for unseen attacks are difficult. We observe the models' classification performance on attack-only traffic arranged in chronological order. As the attack samples increase, unseen attacks may emerge. Fig. 3 shows that the model's ability to identify attacks decreases as the attacks evolve, which urges us to improve the model's generalization. However, complex fingerprinting for normal traffic is useless because it can not improve

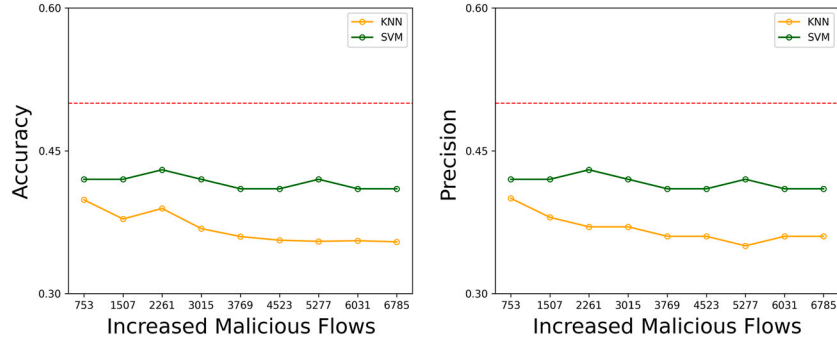


Fig. 3. The attack identification of models on attack traffic.

the classification accuracy but will increase the workload unrelated to identifying attacks. Hence, we utilize a separate tool to detect and isolate normal traffic to eliminate the noise introduced by large amounts of normal traffic. Significantly, after removing normal traffic, the remaining suspicious traffic contains known and unseen attack traffic, which can be further classified precisely. The model does not easily assume unseen attacks as normal, leaving suspicious traffic for further identification, which lays the foundation for an accurate unseen attack classification. Since methods such as SVM, KNN, Random Forest, and DNN can achieve 99% accuracy as mentioned in the research (Ferriyan et al., 2021), a total of four ML models are chosen to verify the effectiveness of this step.

4.2. Attack fingerprinting

4.2.1. Cluster based on host pair

TGPrint clusters the suspicious traffic according to the host's IP pairs because of the attacker's persistent behaviors toward the victim host. After clustering the traffic according to IP pairs, it is possible to analyze the communication interactions between the two hosts and to obtain attack behaviors.

4.2.2. Division based on time window

Attack duration is unstable and varies in different persistent attacks. TGPrint divides attack traffic for each IP pair to adapt to the behaviors' changing over time. When dividing traffic, the longer the time window is, the more traffic is captured, and the more diverse attack behaviors are observed. However, each attack duration is different. Various time windows according to attack types can divide traffic more precisely in the correct type. So we set the time window ratio τ , a parameter that could generate a specific time window for each attack to divide traffic and precisely construct attack graphs. The definition of τ is followed. n_{flows} is the number of flows captured by the time window, and N_{attack} is the number of the corresponding attack flows.

$$\tau = \frac{n_{flows}}{N_{attack}} \quad (2)$$

The details of time window generation are shown in Algorithm 1.

Algorithm 1 Time window generation and use for each attack.

Input: τ , $Label_{attack}$, $Flows_{attack}$

Output: $Graph_{attack}$

```

1 for attack ∈ ATTACKs, generate time window by τ for Labelattack in ATTACKs do
2   lenattack = len(Flowsattack) Time window = lenattack × τ n = lenattack \ Time window
   if count ≤ n then
3     for ith in range (0, n) do
4       Construct attack graphs based on ith time window flows count = count + 1
5     end
6   end
7   return Graphattack
8 end

```

4.2.3. Attack graph construction

High-level attack behaviors with complex interactions among hosts are a natural graph structure where nodes represent hosts and edges represent interaction activities. The similar step, converting traffic flows to graphs (Yao et al., 2019; Xu et al., 2020; Hammann et al., 2019; Pham et al., 2021).

In this paper, an attack type has a set of attack graphs whose nodes are the host's ports, edges indicate the communication behaviors between ports, and edges' attributes describe the behaviors. The edges' attributes indicate the attack communication behaviors through features, which are statistical, temporal, and aggregate features (as described in 4.1.2). The node vectors of a graph are two hosts' ports without IP because the IP address is useless for edge attributes in each graph, which has the same IP pairs after the cluster. Finally, an attack graph is the attack behavioral record between the attack-victim hosts in one time window. In 4.2.4, TGPrint uses these attack graphs to mine hidden attack behavior patterns for attack classification.

4.2.4. Attack fingerprint classification

TGPrint uses attack graphs generate fingerprints as credentials in attack classification. As a result, the attack label is classified according to fingerprints, representing attack graphs. We aim to obtain a representation learning approach $g(G)$ that helps identify the label y_G of an entire graph, $y_G = g(G)$, where a set of graphs $\{G_1, \dots, G_N\} \subseteq G$ and their labels $\{y_1, \dots, y_N\} \subseteq Y$ is given.

We use GNNs to construct the representation learning approach $g(G)$. GNNs can learn the high-dimensional graph structure and extract information from the relationship between hosts and their traffic behaviors (Pham et al., 2021). So we utilize the GNNs to represent the attack graphs, generate attack fingerprints and classify attacks. As discussed in 3.2, GIN (Xu et al., 2018) can produce embedding of the attack graph's nodes, indicating the ports' relationships between attack-victim hosts. GCN (Kipf and Welling, 2016) can extract the feature representation of local host ports and their traffic behaviors and encode them. GAT (Veličković et al., 2017) can leverage *Attention Mechanism* to specify different weights to port nodes and find the more essential neighborhood nodes. On this basis, we construct the model GCAN as a $g(G)$. The model structure is shown in Fig. 4.

GCAN model mainly consists of three layers, GIN layer, GCN layer, and GAT layer. We employ the GIN layer, which has the same size as the input graph. The GCN layer has the input and output size of 128 and the GAT layer has the input and output size of 128. The Top-K Pooling layer is added between each two graph layer because it could select the most important K nodes based on feature values (e.g. degree of nodes, activation of nodes, etc.) to improve the representational power and efficiency. At this point, the vector output from the graph layers is the attack fingerprint. The output of GCAN is transformed from attack fingerprints to attack labels by activation functions. RELU is selected as activation function to increase the computational efficiency because of its simple computational complexity. The final output is a

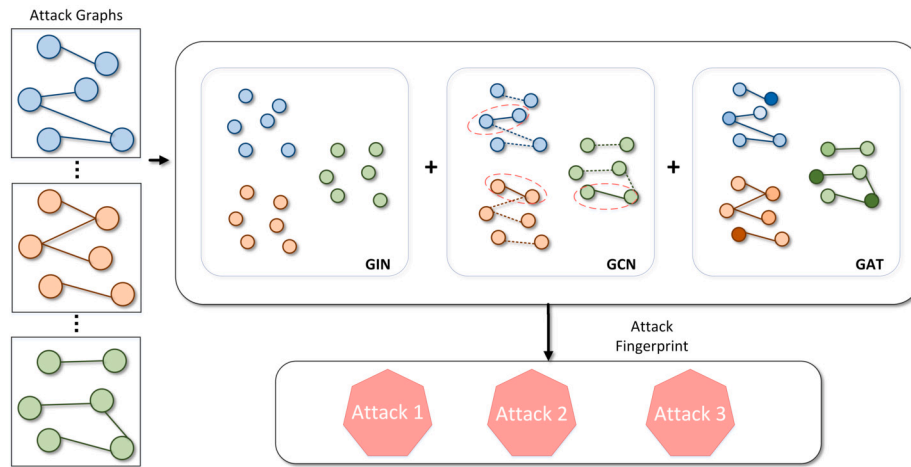


Fig. 4. Architecture of GCAN Model for Attack Classification. Attack graphs are the input. Attack fingerprints are the output of graph layers. Labels are the output of GCAN.

Table 3
Attacks used in CICIDS.

Attack	Tool	Duration
Bruteforce	FTP – Patator	one days
	SSH – Patator	one days
	Web	one days
	xss	one days
DDos	Low Orbit Ion Canon (LOIC) for UDP, TCP,or HTTP requests	two days

4-dimensional attack probability matrix after softmax. The result of attack classification is label with the highest probability in the output.

5. Evaluation and analysis

5.1. Datasets

We use CICIDS and HIKARI 2021 datasets to evaluate TGPrint's effectiveness and superiority in attack classification on encrypted traffic.

5.1.1. CICIDS-2017&CICIDS-2018

CICIDS-2017 (C.I. for Cybersecurity, 2017) and CICIDS-2018 (Sharafaldin et al., 2018) datasets were created by the Canadian Institute for Cyber Security at the University of Brunswick in 2017 and 2018. CICIDS-2017 was proposed for intrusion detection and consisted of several attack scenarios. CICIDS-2018 was produced as an optimized version in the next year. Bruteforce, DoS, Heartbleed, Web Attack, Infiltration, Botnet, and DDoS attacks are produced using publicly available tools and codes. We use Bruteforce and DDoS attacks in experiments (Table 3).

5.1.2. HIKARI-2021

HIKARI-2021 is a dataset containing encrypted synthetic attacks and normal traffic (background and benign) (Ferriyan et al., 2021). Bruteforce, probing, and XMRIGCC CryptoMiner are three chosen attacks because WordPress, Joomla, and Drupal are among the ten most popular open-source CMSs, with a combined market share of almost 50%. Ferriyan made this dataset according to the needed content and the way of attack generated and extracted the features adopted to CICIDS using Zeek.

As well-known datasets, many studies (Chou and Jiang, 2021b) are verified on CICIDS-2017&CICIDS-2018. However, these sets are collected in emulated environments rather than real-world networks and do not have background data. Experiments only on CICIDS lack proof of TGPrint's effectiveness in real networks, so we need more real-world

Table 4
Attacks used in HIKARI-2021.

Attack	Label	Total Flows
Bruteforce	Attack	5584
Bruteforce-XML	Attack	5145
Probing	Attack	23388
XMRIGCC CryptoMiner	Attack	3279

traffic to verify TGPrint. Fortunately, Ferriyan (Ferriyan et al., 2021) propose a new dataset, HIKARI-2021, to solve this flaw, which provides realistic traffic from a real production network. Hence, we conduct extensive experiments on all these datasets. All datasets are sorted in chronological order. We select 80% from each dataset for training and the rest 20% for testing.

5.2. Experimental setup

We implement a prototype of TGPrint in Python using the Scikit-learn and Torch libraries for ML and graph computation. We first confirm the optimal solutions of two parameters, batch size and time window ratio τ on HIKARI2021, which has the same setup and network configurations as CICIDS. Then evaluate our approach with ML models on an attack classification task on three datasets. Afterward, we evaluate the TGPrint's effectiveness to identify unseen attacks. Finally, we compare ML tools during normal traffic denoising and analyze the impact of false positives from the denoising step.

The compared models are outstanding in attack classification on encrypted traffic mentioned in 2.1.2, which are RF, LR, XGBoost, DNN, and LSTM. Their input features are the same 86 features as graph edge attributes. Moreover, Hindy's unseen attack detection method, autoencoder (Hindy et al., 2020), is also compared because it was validated in CICIDS-2017. The conventional performance metrics for multi-class classification, accuracy, precision, and F1-Score are used in this paper to evaluate the performance of models. The specific definitions are as follows, where N is the number of attack types. Thus the evaluation

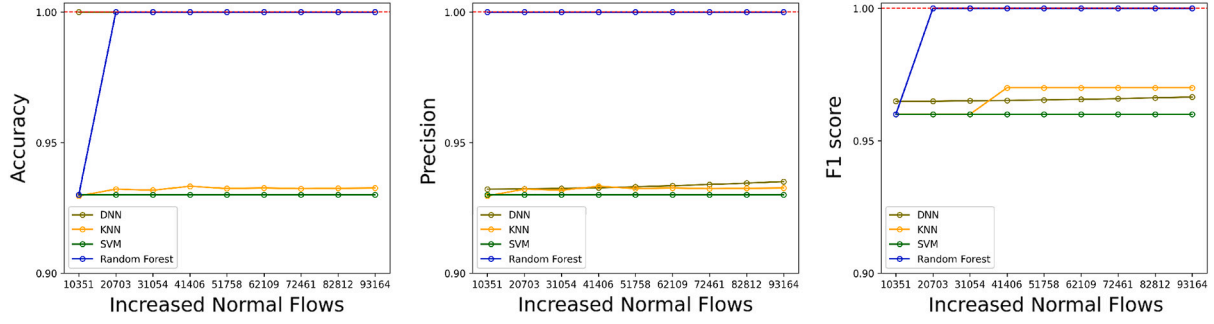


Fig. 5. Model performance on normal-only traffic test sets.

Table 5

Confusion Matrix.

	Actual Positive	Actual Negative
Classified Positive	True Positive, TP	False Positive, FP
Classified Negative	False Negative, FN	True Negative, TN

Table 6

The result of attack traffic denoising.

	Accuracy	Precision	Recall	F1 score
DNN	0.93	0.93	1	0.96
KNN	0.91	0.93	0.91	0.92
SVM	0.89	0.88	0.89	0.89
Random Forest	0.93	1	0.93	0.96

metric is the average of the attack classification performance in all categories (Table 5).

$$Precision = \frac{1}{N} \sum \frac{TP}{TP + FP} \quad (3)$$

$$Accuracy = \frac{1}{N} \sum \frac{TP + TN}{FP + FN + TP + TN} \quad (4)$$

$$F1score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (5)$$

5.3. Analysis of results

5.3.1. Feasibility assessment of attack traffic denoising

To eliminate noise traffic in attack classification, we use the ML model to remove normal traffic before identifying attacks. SVM, KNN, Random Forest, and DNN are used as tools. We first evaluate them on the whole HIKARI 2021. Also, we set a series test set only containing normal traffic and in chronological order to check the generalizability. By comparing the detection accuracy, precision, and recall of the models on these data, we judge the feasibility of the methods for normal traffic denoising. The results on the whole HIKARI 2021 are shown in Table 6 and on series sets are shown in group Fig. 5.

As shown in Table 6, SVM, KNN, Random Forest, and DNN achieve good identification capabilities that accurately distinguish between normal and attack traffic. Except for SVM, the models can achieve accuracy recall and F1 values over 90%. Although results indicate that ML models can accurately distinguish normal and attack traffic in attack traffic denoising, it is still uncertain how well they identify normal traffic.

In group Fig. 5, the x coordinate is the proportion of test samples to all normal traffic. The results demonstrate that the Random Forest performs best, scoring high evaluation metrics in normal traffic, which indicates that the model accurately identifies normal traffic. We believe this is because normal behavior is fixed and constant, so the characteristics of normal traffic are also accessible for the model to learn. Moreover, no decrease is in model recognition over time. Hence, ML models can be used for attack traffic denoising and accurately identifying normal traffic. In TGPrint, we choose Random Forest as the tool.

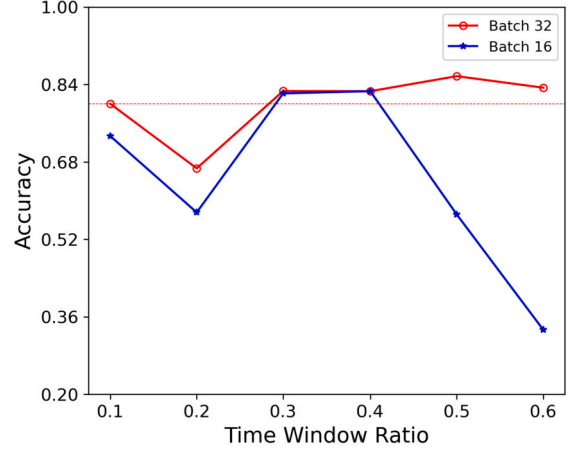


Fig. 6. The accuracy of TGPrint with two batch sizes on attack classification.

5.3.2. Impact of batch size for attack classification

Batch size is a critical parameter in the training process of GCAN since it impacts the experimental results significantly. 16 and 32 are chosen because of their high experiment use. A series of experiments are conducted on the HIKARI2021 dataset. We compare the classification accuracy with different time window ratios τ when the batch size is 16 and 32.

The red dashed line is the baseline for an accuracy of 0.8. The results in Fig. 6 show that accuracy is higher when the batch size is 32 in attack classification. This is because an attack graph contains more behavior information useful for attack classification when the time window rate increases. However, the accuracy decreases as the time window rate exceeds 0.4 when the batch size is 16. The model fails to obtain the complete attack behavior information and quickly falls into local optimal if the batch size is too small for training. Therefore, we chose 32 as the batch size in later experiments.

5.3.3. Impact of time window for attack classification

Time window ratio τ decides the number of attack behaviors in a graph, so it is considered a vital parameter, and we conduct a series of experiments to explore the optimal value. We choose 32 as batch size and accomplish the attack classification task with increasing time window ratio τ to evaluate its impact. The results are shown in Fig. 7.

The red dashed line is the baseline for an accuracy of 0.8. As expected, the classification accuracy for attack fingerprints is affected by the change in the time window ratio τ . The results show that the classification accuracy improves with τ increasing. The increase is non-linear and even decreases in a small range. As the time window ratio increases, more network behaviors are joined, but the final performance can also be affected by improper behavior division which interrupts the integrity of a behavior. Moreover, when the τ exceeds 0.5, the method's performance stabilizes and no longer improves. It is believed that when τ is small, it cannot accurately portray the attack behaviors over time. Con-

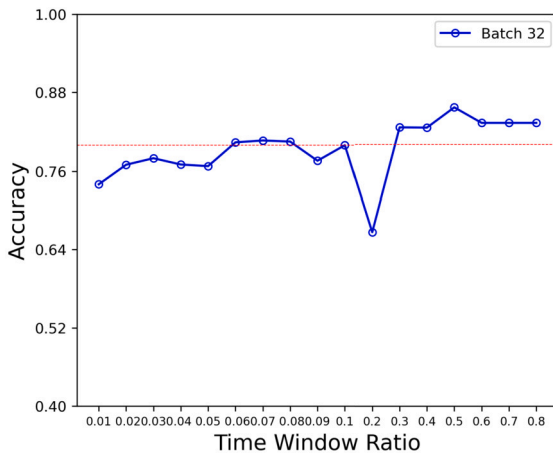


Fig. 7. The accuracy of TGPrint with different Time window rates on attack classification.

Table 7

Performances of different methods in attack classification on HIKARI 2021.

Methods	Accuracy	Precision	F1 score
Random Forest	0.32	0.58	0.41
Logistic Regression	0.32	0.33	0.48
XGBoost	0.24	0.5	0.42
DNN	0.12	0.25	0.04
LSTM	0.2	0.48	0.27
TGPrint	0.86	0.99	0.89

Table 8

Performances of different methods in attack classification on CICIDS.

Methods	Accuracy	Precision	F1 score
Random Forest	0.29	0.14	0.22
Logistic Regression	0.47	0.23	0.32
XGBoost	0.52	0.27	0.36
DNN	0.22	0.69	0.45
LSTM	0.16	0.64	0.79
TGPrint	0.67	0.99	0.80

versely, when the time window is long enough, an attack graph may not provide more behavioral information because the attacker will try again when they encounter an obstacle. Therefore too much traffic only adds an increased burden on the training of attack graphs without help to improve the classification performance.

Besides, the results also show that although τ affects the model effectiveness, choosing a shorter time window may result in higher gains, i.e., faster identification of attack type, compared to the loss in performance. In practice, the users can configure this parameter according to the actual requirements to achieve optimal results.

5.3.4. Performance of attack classification

Attack classification on encrypted traffic is the primary purpose of this paper, so we conduct experiments on three datasets and compare TGPrint with the other five ML models on the performance of attack classification. The experimental results are shown in Table 7 and Table 8.

The results prove that TGPrint achieves the highest scores in the evaluation metrics, especially in the precision of attack classification. It owes to the fact that an attack graph extracts the behaviors of each attack, and GCAN obtains the crucial attack behavior patterns as a fingerprint to classify attacks. Nevertheless, other models perform poorly on both datasets, even with the same features. Models perform inadequately because they fail to grasp the crucial features. The previous

Table 9

Classification results of different model structures on the HIKARI 2021.

Layers	Accuracy	Precision	F1 score
GIN+GCN	0.83	0.99	0.83
GIN+GAT	0.83	0.99	0.83
GCAN	0.86	0.99	0.89

Table 10

Classification results of different model structures on the CICIDS.

Layers	Accuracy	Precision	F1 score
GIN+GCN	0.46	0.99	0.61
GIN+GAT	0.54	0.87	0.62
GCAN	0.67	0.99	0.8

studies focus on the classification ability based on flow level but ignore that attack is continuous and traffic is associated with each other.

To show the representation ability of our method for attacks, we draw the scattergrams to observe the distribution differences before and after TGPrint. The results are shown in Fig. 8. The distribution of the original traffic flows overlaps, but after TGPrint, the attack flows are mapped to another concentrated area and can be distinguished clearly. This result strongly proves the rationality and validity of TGPrint in design. On the one hand, attack fingerprint adds behaviors of attack-victim hosts over time, compensating for the limited features to some extent. On the other hand, the three graph network layers in GCAN obtain the critical attack behavior patterns from the information and relationships so that attacks can finally be precisely distinguished. As the GIN layer produces embedding of the attack graph's nodes, which is necessary for graph representation, we use the GIN layer respectively with GCN and GAT layers to conduct classification experiments on HIKARI 2021. The results in Table 9 and Table 10 demonstrate that the structure of GCAN is reasonable, which better captures attack behavior patterns.

5.3.5. Performance of unseen attack classification

Whether a model classifies accurately facing unseen attack represents the ability to counter unknown risks. We choose DDoS and brute-force as the attack labels having new attack vectors and perform a series of unseen attack classification experiments. We constructed two attack scenarios: one unseen attack and multiple unseen attacks. We compare TGPrint with four ML models having good performance in attack classification and Hindy's study (Hindy et al., 2020). They also verified the autoencoder on unseen attack detection on CICIDS, and we directly quote the results from the paper. The results are recorded in Table 11 and Table 12.

Obviously, except for TGPrint, all other methods have a weak ability to identify unseen attacks, even though they can obtain high evaluation metric scores in individual scenarios. In the HIKARI2021 dataset, TGPrint can identify BruteForce implemented by XML based on the fingerprints of BruteForce implemented by HTTP. In CICIDS datasets, TGPrint can successfully identify unseen BruteForce based on four attack vectors, SSH, FTP, Web, and XSS, separately. In the multiple unseen attacks scenario, TGPrint could identify them with an accuracy of more than 0.6, up to 0.71. Although the evaluation metrics are lower than XGBoost regarding unseen attacks combining XSS and LOIC, XGBoost's identification scores in all other scenarios are much lower than TGPrint's. Hence, it can be concluded that TGPrint can identify unseen attacks accurately.

TGPrint can identify unseen attacks because GCAN generates attack fingerprints from attack graphs and reveal fundamental techniques of the attack. Different attack vectors have the same attack tactics. TGPrint grasps these tactics by crucial attack behavior patterns generated from

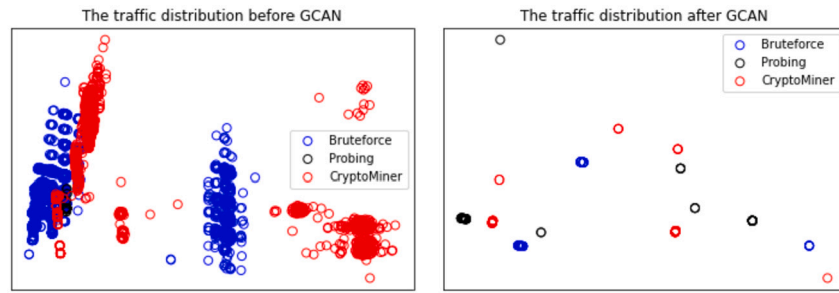


Fig. 8. The distribution of attack traffic before and after TGPrint.

Table 11
Performance on unseen attacks in HIKARI 2021.

Attack	New Vector	Method	Accuracy	Precision	F1 score
Bruteforce	XML	TGPrint	0.8	0.67	0.5
		Random Forest	0.72	0.52	0.6
		Logistic Regression	0.5	0.5	0.5
		XGBoost	0.5	1	1
		DNN	0.1	0.25	0.01
		LSTM	0.1	0.41	0.26

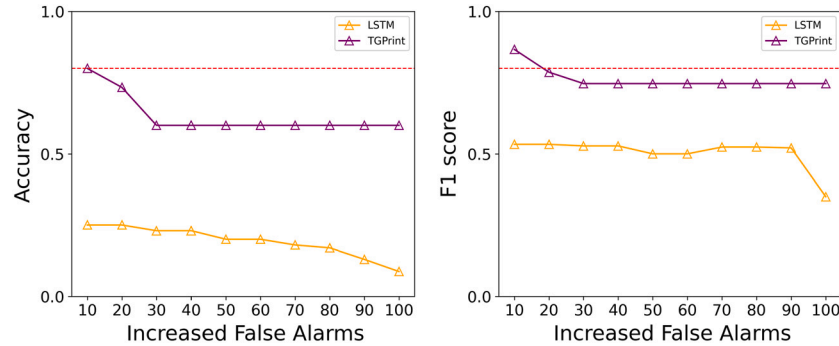


Fig. 9. The classification performance of the models decrease as the number of false positives increases.

attack graphs to classify unseen attacks. GCAN's representation ability could successfully obtain these attack behavior patterns. Hence, TGPrint is a practical approach to tackling unseen attacks.

5.3.6. Impact of false positive alarms

Although ML models can accurately identify normal traffic, they still have false positives, impacting subsequent classification tasks. For example, if normal traffic is identified as attack traffic, these false positives impact the construction of attack graphs. In attack fingerprinting, normal is not the target label, so where false positives will be classified significantly impacts the model's classification ability. As TGPrint clusters traffic flows according to host pair, false positives are added in attack graphs to generate fingerprints to classify attacks. Hence, we add normal flows in the attack graph as false positives and conduct attack classifications on HIKARI 2021 to test TGPrint's resistance to false positives. We choose LSTM as a comparison algorithm because it also has a time window. Fig. 9 depicts results to classify attacks with false positives.

The red dashed line is the baseline for an accuracy and F1 score of 0.8. As expected, the models' accuracy and F1 score decrease significantly as false positives increase. TGPrint's classification performance decreases at the beginning but stabilizes after false positives increase to a certain level. LSTM's classification performance decreases with false positives added. These results show that TGPrint is feasible to weaken false positives as it grasps crucial attack behavior patterns although

existing false positives. In conclusion, TGPrint is a stable method regarding classification capability and robustness to false positives.

Although TGPrint is robust in false positives, the degradation of classification performance leads to more losses. Fortunately, attack traffic denoising performance can be improved, reducing the crippling effect of false positives in attack classification and allowing the model to maintain high accuracy.

5.3.7. Attack discussion

The classification performance of TGPrint varies across different attack types. The optimal outcomes are presented in Table 13. Probing and Bruteforce attacks are accurately recognizable, identification of DDoS is significantly improved, and CryptoMiner encounters challenges in recognition due to limited sample availability (Table 4). Resampling was not employed, as the difference in the count of distinct attack samples remains minimal. The small number of CryptoMiner traffic samples results in a constraint on the number of attack traffic flow-generated graphs. Consequently, TGPrint's ability to sufficiently learn CryptoMiner's attack patterns is hindered, leading to reduced classification performance for CryptoMiner attacks. Conversely, attacks with ample samples and diverse attack vectors, like Probing and Bruteforce, achieve accurate recognition due to more effective learning of attack patterns, resulting in improved classification accuracy.

Furthermore, TGPrint exhibits proficiency in categorizing persistent attacks. The four attacks listed in the table, Bruteforce, DDoS, probing, and CryptoMiner, demonstrate consistent attack behaviors over time.

Table 12
Performance on unseen attacks in CICIDS.

Attack	New Vector	Method	Accuracy	Precision	F1 score
Bruteforce	SSH	TGPrint	1	1	1
		Random Forest	0	0	0
		Logistic Regression	1	1	1
		XGBoost	0.12	0.06	0.1
		DNN	1	1	1
		Hindy (Hindy et al., 2020)	0.81	-	-
Bruteforce	FTP	TGPrint	1	1	1
		Random Forest	0	0	0
		Logistic Regression	0.02	0.01	0.2
		XGBoost	0	0	0
		DNN	1	1	1
		Hindy (Hindy et al., 2020)	0.20	-	-
Bruteforce	XSS	TGPrint	1	1	1
		Random Forest	0.01	0	0.01
		Logistic Regression	0.96	0.48	0.49
		XGBoost	0.43	0.14	0.2
		DNN	1	1	1
Bruteforce	Web	TGPrint	1	1	1
		Random Forest	0.13	0.04	0.08
		Logistic Regression	0.45	0.15	0.21
		XGBoost	0.36	0.09	0.13
		DNN	1	1	1
DDoS+ Bruteforce	SSH+ LOIC	TGPrint	0.71	0.5	0.67
		Random Forest	0	0	0
		Logistic Regression	0.63	0.33	0.33
		XGBoost	0.37	0.33	0.19
		DNN	0.63	0.5	0.39
DDoS+ Bruteforce	Web+ LOIC	TGPrint	0.6667	0.67	0.8
		Random Forest	0	0	0
		Logistic Regression	0.06	0.16	0.22
		XGBoost	0.08	0.21	0.14
		DNN	0.13	0.5	0.11
DDoS+ Bruteforce	XSS+ LOIC	TGPrint	0.67	0.677	0.8
		Random Forest	0.02	0.16	0.22
		Logistic Regression	0.04	0.32	0.32
		XGBoost	0.98	0.76	0.84
		DNN	0.04	0.5	0.04
DDoS+ Bruteforce	XSS+ HOIC	TGPrint	0.6667	0.6667	0.8
		Random Forest	0.01	0.15	0.2
		Logistic Regression	0.02	0.31	0.02
		XGBoost	0.96	0.38	0.42
		DNN	0.03	0.5	0.03
Average		TGPrint	0.84	0.81	0.88

Table 13
Optimal Performance of TGPrint in Different Attacks.

Attack	Accuracy/top 1	Precision/top 1	F1 score/top 1
Bruteforce	0.8	0.99	0.8
DDoS	0.73	0.5	0.67
Probing	0.99	0.99	0.99
CryptoMiner	0.8	0.5	0.5

Despite variations in attack implementations, their persistent network behaviors on victim hosts are captured in traffic records. By constructing attack graphs and learning patterns within defined time windows, TGPrint excels in identifying attacks characterized by persistent network behaviors.

However, TGPrint's efficacy in dealing with non-persistent attacks remains uncertain. Learning attack patterns from a single-flow attack graph poses challenges for TGPrint. The model struggles to discern attack patterns in such scenarios. Subsequent research endeavors are to delve deeper into TGPrint's classification capabilities, thereby providing a more comprehensive understanding of its potential.

6. Limitation

This paper proposes TGPrint to classify attacks in encrypted traffic. However, there are two limitations existing.

Feasibility in real networks. Although TGPrint is verified on datasets from a real network environment, how feasible it would work in a real network still needs to be determined. More practical factors, such as identification efficiency, resource occupancy, and network structure, should be considered in a real network. In TGPrint, the time window rate τ also affects the performance in real networks. These effects cannot be judged on experiment results on datasets alone. In future research, TGPrint should implement in real-time in real networks. Meanwhile, the calculation should be carried out in time complexity and space complexity to verify the performance of TGPrint.

Incomplete attack types. The types of attacks are incomplete and limited in datasets, and more attack types are in real networks. This paper proves TGPrint's classification performance on collected attacks, but its performance on other attacks in real networks needs further discussion. TGPrint identifies bruteforce, infiltration, mining, and DDoS, but if a worm occurs, TGPrint may misdiagnose because there is no

corresponding attack fingerprint. While attacks will not be classified as normal, there may be misclassifications between different attacks. Therefore, the fingerprint database generated by TGPrint decides the performance of attack classification and affects network security. An incomplete attack fingerprint database will give wrong judgment in attack classification, which hinders subsequent response to security events. In future work, we need to collect more encrypted attack traffic and enrich the attack fingerprint database to classify accurately.

Universality of different encryption methods. While TLS stands as the predominant encryption protocol, alternatives like Tor and VPN persist. Concealed attacks within these methods necessitate attention. Consequently, the capacity to classify attacks within different encryption methods remains pertinent. In HIKARI 2021, encryption utilizes TLS-based HTTPS, while in CICIDS, manual encryption conceals payload data. TGPrint's classification performance was assessed on both encryption methods, but the findings fall short of demonstrating its universal applicability to encrypted traffic. Thus, further investigation is required to address attack classification in diverse encryption-based traffic scenarios.

7. Conclusion

This paper presents TGPrint, a novel attack classification approach that obtains crucial attack behavior patterns and deals with unseen attack classification. We first use Random Forest for attack traffic denoising to remove the normal traffic. Then, we propose a novel parameter time window ratio τ to divide a specific duration for each attack to elaborate attack graphs precisely. Finally, we utilize GCAN to mine and grasp the crucial behavior patterns from attack graphs to generate fingerprints and classify attacks. Extensive experiments are conducted on three datasets to verify TGPrint. The experimental results show that TGPrint achieves a precision of 99% in attack classification on encrypted traffic, an average higher than other ML methods of 50%. It also classifies unseen attacks with an average accuracy of over 80% and has a solid robustness to false positives.

CRediT authorship contribution statement

Leiqi Wang: Writing – original draft preparation, Conceptualization, Methodology.
Xiu Ma: Methodology.
Ning Li: Software.
Qiujian Lv: Writing – reviewing & editing.
Yan Wang: Supervision.
Weiying Huang: Resources, Validation.
Haiyan Chen: Reviewing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Abuse, 2022. abuse.ch. ssl blacklist. <https://ssllbl.abuse.ch/>.
- Alqudah, N., Yaseen, Q., 2020. Machine learning for traffic analysis: a review. *Proc. Comput. Sci.* 170, 911–916.
- Anderson, B., McGrew, D., 2017. Machine learning for encrypted malware traffic classification: accounting for noisy labels and non-stationarity. In: *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1723–1732.
- Anderson, B., Paul, S., McGrew, D., 2018. Deciphering malware's use of tls (without decryption). *J. Comput. Virol. Hacking Tech.* 14 (3), 195–211.
- Arora, A., Garg, S., Peddoju, S.K., 2014. Malware detection using network traffic analysis in Android based mobile devices. In: *2014 Eighth International Conference on Next Generation Mobile Apps, Services and Technologies*. IEEE, pp. 66–71.
- Bar, R., Hajaj, C., 2022. Simcse for encrypted traffic detection and zero-day attack detection. *IEEE Access*.
- C.I. for Cybersecurity, 2017. Intrusion detection evaluation dataset (cic-ids2017). <https://www.unb.ca/cic/datasets/ids-2017.html>.
- Chen, F., Wang, Y.-C., Wang, B., Kuo, C.-C.J., 2020. Graph representation learning: a survey. In: *APSIPA Transactions on Signal and Information Processing*, vol. 9.
- Chou, D., Jiang, M., 2021a. A survey on data-driven network intrusion detection. *ACM Comput. Surv.* 54 (9), 1–36.
- Chou, D., Jiang, M., 2021b. A survey on data-driven network intrusion detection. *ACM Comput. Surv.*
- Cisco, 2019. Encrypted traffic analytics. White paper. <https://www.cisco.com/c/en/us/solutions/enterprise-networks/enterprise-network-security/eta.html>.
- Dierks, T., 1999. The tls protocol version 1.0. <https://datatracker.ietf.org/doc/html/rfc2246>.
- Duessel, P., Gehl, C., Flegel, U., Dietrich, S., Meier, M., 2017. Detecting zero-day attacks using context-aware anomaly detection at the application-layer. *Int. J. Inf. Secur.* 16 (5), 475–490.
- Erman, J., Mahanti, A., Arlitt, M., Williamson, C., 2007. Identifying and discriminating between web and peer-to-peer traffic in the network core. In: *Proceedings of the 16th International Conference on World Wide Web*, pp. 883–892.
- Ferriyan, A., Thamrin, A.H., Takeda, K., Murai, J., 2021. Generating network intrusion detection dataset based on real and encrypted synthetic attack traffic. *Appl. Sci.* 11 (17), 7868.
- FRANTISEK S, S.G., 2017. Machine learning for network https analysis. In: *BRUCon Conference*.
- Gu, G., Perdisci, R., Zhang, J., Lee, W., 2008. Botminer: Clustering Analysis of Network Traffic for Protocol- and Structure-Independent Botnet Detection.
- Hammann, S., Radomirović, S., Sasse, R., Basin, D., 2019. User account access graphs. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1405–1422.
- Hindy, H., Atkinson, R., Tachtatzis, C., Colin, J.-N., Bayne, E., Bellekens, X., 2020. Utilising deep learning techniques for effective zero-day attack detection. *Electronics* 9 (10), 1684.
- Kipf, T.N., Welling, M., 2016. Semi-supervised classification with graph convolutional networks. *ArXiv preprint. arXiv:1609.02907*.
- Koch, R., Golling, M., Rodosek, G.D., 2014. Behavior-based intrusion detection in encrypted environments. *IEEE Commun. Mag.* 52 (7), 124–131.
- Lopez-Martin, M., Carro, B., Sanchez-Esguevillas, A., Lloret, J., 2017. Network traffic classifier with convolutional and recurrent neural networks for Internet of things. *IEEE Access* 5, 18042–18050.
- Luo, Z., Xu, S., Liu, X., 2020. Machine learning-based scheme for detecting tls malicious encrypted traffic. *J. Netw. Inf. Secur.* 6 (1), 77–83.
- Madhukar, A., Williamson, C., 2006. A longitudinal study of p2p traffic classification. In: *14th IEEE International Symposium on Modeling, Analysis, and Simulation*. IEEE, pp. 179–188.
- Mohamad Tahir, H., Hasan, W., Md Said, A., Zakaria, N.H., Katuk, N., Kabir, N.F., Omar, M.H., Ghazali, O., Yahya, N.I., 2015. Hybrid Machine Learning Technique for Intrusion Detection System.
- Nychis, G., Sekar, V., Andersen, D.G., Kim, H., Zhang, H., 2008. An empirical evaluation of entropy-based traffic anomaly detection. In: *Proceedings of the 8th ACM SIGCOMM Conference on Internet Measurement*, pp. 151–156.
- Park, J.-S., Yoon, S.-H., Kim, M.-S., 2013. Performance improvement of the payload signature based traffic classification system using application traffic locality. *J. Korean Inst. Commun. Inf. Sci.* 38 (7), 519–525.
- Pham, T.-D., Ho, T.-L., Truong-Huu, T., Cao, T.-D., Truong, H.-L., 2021. Mappgraph: mobile-app classification on encrypted network traffic using deep graph convolution neural networks. In: *Annual Computer Security Applications Conference*, pp. 1025–1038.
- Popoola, S.I., Ande, R., Adebisi, B., Gui, G., Hammoudeh, M., Jogunola, O., 2021. Federated deep learning for zero-day botnet attack detection in iot-edge devices. *IEEE Int. Things J.* 9 (5), 3930–3944.
- Richman, S., 2022. This new covert channel uses ssl/tls handshakes. <https://www.extrahop.com/company/blog/2018/stop-ssl-tls-exfil/>.
- Roques, O., Maffei, S., Cova, M., 2019. Detecting malware in tls traffic. In: *The IEEE Conference on Local Computer Networks 30th Anniversary (LCN'05)*.
- Sarhan, M., Layeghy, S., Gallagher, M., Portmann, M., 2021. From zero-shot machine learning to zero-day attack detection. *ArXiv preprint. arXiv:2109.14868*.
- Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A., 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: *ICISp*, vol. 1, pp. 108–116.
- Shen, M., Wei, M., Zhu, L., Wang, M., Li, F., 2016. Certificate-aware encrypted traffic classification using second-order Markov chain. In: *2016 IEEE/ACM 24th International Symposium on Quality of Service (IWQoS)*. IEEE, pp. 1–10.
- Suricata, 2010. Open information security foundation. <https://suricata.io/>.

- Torroledo, I., Camacho, L.D., Bahnsen, A.C., 2018a. Hunting malicious tls certificates with deep neural networks. In: *Proceedings of the 11th ACM Workshop on Artificial Intelligence and Security*, pp. 64–73.
- Torroledo, I., Camacho, L.D., Bahnsen, A.C., 2018b. Hunting malicious tls certificates with deep neural networks. In: *The 11th ACM Workshop*.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y., 2017. Graph attention networks. *ArXiv preprint. arXiv:1710.10903*.
- Xu, K., Hu, W., Leskovec, J., Jegelka, S., 2018. How powerful are graph neural networks? *ArXiv preprint. arXiv:1810.00826*.
- Xu, X., Du, X., Zeng, Q., 2020. Attacking graph-based classification without changing existing connections. In: *Annual Computer Security Applications Conference*, pp. 951–962.
- Yao, Y., Su, L., Lu, Z., Liu, B., 2019. Stdeephgraph: spatial-temporal deep learning on communication graphs for long-term network attack detection. In: *2019 18th IEEE International Conference on Trust, Security and Privacy in Computing and Communications/13th IEEE International Conference on Big Data Science and Engineering (TrustCom/BigDataSE)*. IEEE, pp. 120–127.
- Zhai, M., Zhang, X., Zhao, B., 2020. Survey of encrypted malicious traffic detection based on deep learning. *J. Netw. Inf. Secur.* 6 (3), 66–77.

Leiqi Wang received bachelor from University of Science and Technology Beijing, in 2018. She is presently pursuing the Ph.D. degree at School of Cyber Security, University of Chinese Academy of Sciences. Her current research interests include deep learning and malicious network traffic detection.

Xiu Ma is presently pursuing the Ph.D. degree at School of Cyber Security, University of Chinese Academy of Sciences. Her current research interests include network attack detection and analysis.

Ning Li received master's degree from Beijing University of Posts and Telecommunications, Beijing, China. He is now an associate senior engineer in the institute of information engineering, Chinese Academy of Sciences. His research interests include anomaly detection, user behavior analysis, and network security operation and maintenance technology.

Qiujian Lv received Ph.D. degree from information and communication engineering from the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications. She is now an associate senior engineer in the institute of information engineering, Chinese Academy of Sciences. Her research interests include anomaly detection and data mining.

Yan Wang received master's degree from Beijing University Of Technology, Beijing, China. She is now a senior engineer in the institute of information engineering, Chinese Academy of Sciences. Her research interests include anomaly detection and network security situational awareness.

Weiqing Huang is a research professor, PhD supervisor of Institute of Information Engineering, Chinese Academy of Sciences. He is also a professor in the School of Cyber Security, University of Chinese Academy of Sciences. His research interests include signal processing theory and technology, electromagnetic acousto-optical detection and protection.

Haiyan Chen received Ph.D. degree from School of Chemistry and Chemical Engineering, University of Pisa, Italy. Her research interests include biodegradation studies of pollutants in environmental media and ecological remediation of pollutants.